

Different approaches to using Git

- Console
- CMD

Intro:

Git helps for:

- Track file changes
- Collaborate with teams
- Maintain version history
- Manage branches

Configuration:

git config --global [user.name](#) "YourName"

git config --global user.email "[your@email.com](#)"

Check by

git config --list

Create a local repository:

mkdir folderName

cd folderName

git init

Creates a .git file to track changes

Create a file

touch [myapp.py](#)

git add [myapp.py](#)

Adds only specified file into staging area

git commit -m "Message"

Remote repositories:

- GitHub
- GitLab
- Bitbucket

git remote add origin remoteRepoLink

git remote -v

git push -u origin main

git push

After first push, only this required

git diff

View file changes

git pull

Fetch + merge remote changes

git clone

git branch feature1

git checkout feature1

git checkout -b feature1

Shortcut

git checkout main

git merge feature1

git branch -d feature1

Basic Git Workflow:

Working directory -> Staging Area (add) -> Local Repo (commit) -> Remote Repo(push)

SQL

DDL (focuses on the Table structures) , DML (focuses on the Data inside tables) ,
DCL (Data Control L, for giving permissions using Grant and Revoke),
TCL (Transactional Control L)

Q: Difference between DELETE, TRUNCATE and DROP.

A: DELETE used to delete specific rows in a table
TRUNCATE used to delete all rows in a table, preserving the structure
DROP used to delete table structure altogether with the data in it

NoSQL

Not only SQL

Comes under Big Data technology

Limitation of RDBMS:

- Document storage (Does not deal with big data)
- Fixed columns or structure
- Cannot store unstructured data

Different types of DBs under NoSQL

- Key-value : Redis, Riak
- Columnar : Hbase, Cassandra
- Document : mongodb
- Graphical : NE04

MongoDB

Embedded documents instead of joins and foreign keys

Flexible and dynamic schema

Collections (JSON document)

Horizontal scaling

Stored as **BSON** (Binary JSON) documents

Hierarchy:

Server

-> Database

-> Collection

-> Document (JSON)

Example:

```
{
  "_id": 1,
  "name": "Amit",
  "dept": "IT",
```

```

        "Salary":750000
    }
}
use EmployeeDB
switched to db EmployeeDB
db.createCollection("employees")

```

```

db.employees.insertOne({
  name:"Priya",
  department:"HR",
  salary:600000,
  experience:3
})

```

```

db.employees.insertMany([ { }, { } ])

```

```

db.employees.updateOne(
  {name:"Aadharsh"},
  {$set:{dept:"Data"}}
);

```

```

db.employees.updateMany(
  {dept:"Data"},
  {$inc:{salary:50000}}
);

```

\$set Update field value
 \$inc Increment value
 \$push Add element to array
 \$unset Remove field

```

db.employees.deleteOne({name:"Sneha"})

```

```

db.employees.deleteMany( {dept: "Finance"} );

```

```

db.employees.find()

```

```

db.employees.find({dept:"Data"})

```

```

db.employees.find({salary:{$gte:425000}})

```

```

db.employees.aggregate([
  {$group: { _id: "$dept", avgSal: {$avg:"$salary"}}}
])

```

```

db.employees.aggregate([
  {$group: { _id: "$dept", totalEmps: {$sum:1}}}
])

```

```

db.employees.aggregate([
  {$sort: {salary:-1}}
]) # 1 for ascending

```

```
db.employees.createIndex({dept:1,salary:-1})
db.employees.getIndexes()
```

Mini Project:

- Employee collection
- Insert 50 records
- Perform:
 - Salary analysis
 - Department headcount
 - Top 5 highest paid employees
 - Create index on salary
 - Run aggregation pipeline

Total salary:

```
db.Employee.aggregate([
  {$group:{_id:null,totalSalary:{$sum:"$salary"}}}
])
```

Department headcount:

```
db.Employee.aggregate([
  {$group:{_id:"$dept",totalEmps:{$sum:1}}}
])
```

Top 5 highest paid employees:

```
db.Employee.aggregate([
  {$sort : {salary: -1}},
  {$limit: 5}
])
```

Create index on salary:

```
db.Employee.createIndex({salary:1})
salary_1
```

Aggregation pipeline on Departments having more than 4 employees

```
db.Employee.aggregate([
  {$group : { _id: "$dept", count : {$sum : 1}}},
  {$match : { "count" : {$gte : 5}}}
])
```

Aggregation by grouping departments for department salaries

```
db.Employee.aggregate([
  {$group : { _id: "$dept", count : {$sum : 1}, totalSalary: {$sum : "$salary"}}},
  {$match : { "count" : {$gte : 5}, "totalSalary" : {$gt: 423000}}}
])
```

SCALA

Datatypes : Val and Var

Type inference decides the datatype, just like in python

Val is immutable, like final in Java

Var is mutable

```
var num = "10"
Integer.parseInt(num)
val res10: Int = 10
```

num = 20 will fail because scala is statically typed language (use val or var), type remains same for the entire scope

```
val x = {val a = 10; val b = 50; b-a;}
val x: Int = 40
// {} is list of expression, result is also expression, value is the last expression
```

```
val s = if (x > 0 && x < 6) "positive" else 0
val s: Any = 0
// The any datatype because the results are of mixed datatypes
```

Var args = "Hello"

Var i = 0

```
while( i < args.length)
| {
| println(args(i))
| i+=1
| }
```

H
e
l
l
o

i++ will not work in Scala

```
args.foreach(arg => println(arg))
```

H
e
l
l
o

```
args.foreach(println)
```

H
e

|
|
o

```
for(i <- 1 to 5) // can give args.length instead of 5
  | print(i)
12345
```

```
for(i <- 0 until 10) // exclusive of 10 here
  | print(i)
0123456789
```

```
var i = 0; var j = 0
  var i: Int = 0
  var j: Int = 0
```

```
for(i<-1 to 3;j<-1 to 3) // Multiple generator
  | println((i*10)+j)
11
12
13
21
22
23
31
32
33
```

```
for(i<-1 to 3;j<-1 to 3; if i==j)
  | println((i*10)+j)
11
22
33
```

```
for(i<- 1 to 3; x = 4-i;j<- x to 3) println(10*i+j)
13
22
23
31
32
33
```

// Vector using yields

```
val x = for(i<-1 to 20) yield i*2.5
      val x: IndexedSeq[Double] = Vector(2.5, 5.0, 7.5, 10.0, 12.5, 15.0, 17.5, 20.0, 22.5,
25.0, 27.5, 30.0, 32.5, 35.0, 37.5, 40.0, 42.5, 45.0, 47.5, 50.0)
```

```
for(i<- x) println(i)
```

2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
22.5
25.0
27.5
30.0
32.5
35.0
37.5
40.0
42.5
45.0
47.5
50.0

// Function definition

```
def area(radius: Int): Double = (3.14*radius*radius)
  def area(radius: Int): Double
```

```
def factorial(n : Int): Int = if(n == 0) 1 else n * factorial(n-1)
```

// Function call

```
area(7)
  val res54: Double = 153.86
```

Assignment

Q: Write the function of area of rectangle

```
def rectArea(b: Int, w:Int): Int = (w * b)
  def rectArea(b: Int, w: Int): Int
```

```
rectArea(4,7)
  val res60: Int = 28
```

Q: Write the function to check whether a number is Prime or not

```
def checkPrime(value: Int): Boolean = {res = true;
  | for(i <- 2 to value-1)
  | if (value % i == 0)
  | res = false;
  | res;
  | }
```



```
def checkPrime(value: Int): Boolean
```

// Default arguments

```
def concatStr(arg1:String, arg2:String = "Ajeeb", arg3:String ="Kunhuty"): String =  
(arg1+arg2+arg3)  
  | concatStr("Hi")
```

```
def concatStr(arg1: String, arg2: String, arg3: String): String  
val res67: String = HiAjeebKunhuty
```

// Procedures

(Special functions in Scala that doesn't return any value)

```
def rect_area(b:Int, w:Int) {val area = b*w; println(area)}  
var x = rect_area(3,7)  
  21  
var x: Unit = ()
```

// Collections

Array, ArrayBuffers, Maps, Tuples, Lists

```
val n = new Array[Int](10)  
  val n: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

```
val st = Array("Hello", "World")  
  val st: Array[String] = Array(Hello, World)
```

```
import scala.collection.mutable.ArrayBuffer
```

```
val a = ArrayBuffer[Int]()  
  val a: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer()
```

```
a += 1  
  val res76: a.type = ArrayBuffer(1)
```

```
a += (2,3,4)  
  warning: method += in trait Growable is deprecated (since 2.13.0): Use `++=` aka  
  `addAll` instead of varargs `+=`; infix operations with an operand of multiple args will be  
  deprecated  
  val res77: a.type = ArrayBuffer(1, 2, 3, 4)
```

```
a ++= Array(5,6,7)  
  val res78: a.type = ArrayBuffer(1, 2, 3, 4, 5, 6, 7)
```

```
a.foreach(println)  
  1  
  2
```

3
4
5
6
7

```
a.trimEnd(2) // removes two elements from end
a.insert(2, 9) // inserts 9 at index 2
a.remove(2) // removes element at index 2
a.remove(2,3)// removes 3 elements from index 2
```

```
a.max
    val res15: Int = 7
```

```
a.sum
    val res16: Int = 30
```

```
val b = Array(1,7,2,9)
    val b: Array[Int] = Array(1, 7, 2, 9)
```

```
scala.util.Sorting.quickSort(b)
```

```
b.mkString("****")
    val res18: String = 1**2**7**9
```

// Mapping

(Immutable)

```
val mapping = Map("Ajeeb"->"AJK", "Rithul"->"AK")
    val mapping: scala.collection.immutable.Map[String,String] = Map(Ajeeb -> AJK,
Rithul -> AK)
```

```
val x = mapping("Ajeeb")
    val x: String = AJK
```

```
val z = mapping.getOrElse("AJ",0)
    val z: Any = 0
```

(Mutable)

```
val mutableMapping = scala.collection.mutable.Map("AJK"->"Ajeeb","AK"-
>"Rithul")
    val mutableMapping: scala.collection.mutable.Map[String,String] = HashMap(AK ->
Rithul, AJK -> Ajeeb)
```

```
val y = mutableMapping("AJK")
    val y: String = Ajeeb
```

```
mutableMapping += ("Ajay" -> "Sharma")
mutableMapping -= "AK" // removes key value pair
```

// Tuples

```
val a = (1,4,"Bob","Jack")
      val a: (Int, Int, String, String) = (1,4,Bob,Jack)

a._2 // accessing tuple elements, offsets starts with 1
val res23: Int = 4

"New Delhi Index".partition(_.isUpper)
val res24: (String, String) = (NDI,ew elhi ndex)
```

// List

```
List[Int] = List(1, 2)

2::List(4,5)
val res25: List[Int] = List(2, 4, 5)

def sum(l:List[Int]):Int = {if(l==Nil) 0 else l.head+sum(l.tail)}
def sum(l: List[Int]): Int

val y = sum(lst)
val y: Int = 3

var lst = List(1,2,3,4,5)
var lst: List[Int] = List(1, 2, 3, 4, 5)

sum(lst) // self defined function instead of lst.sum
val res29: Int = 15

lst.tail
val res30: List[Int] = List(2, 3, 4, 5)

lst.head
val res31: Int = 1
```

Assignments